

Einkaufsliste App Reflexion

Von:

Kiera Smith

&

Samuel Trajano

Einführung

Wie bereits erläutert: Aufgrund eines Missverständnisses bezüglich Benotung, haben wir uns vollständig auf die UI/UX fokussiert und den Code teil vernachlässigt. Die Applikation haben wir mittels Prompts von Figma stylen lassen und anschliessend die .make Datei einem Coding Agenten übergeben um ein React JS Projekt daraus zu erstellen.

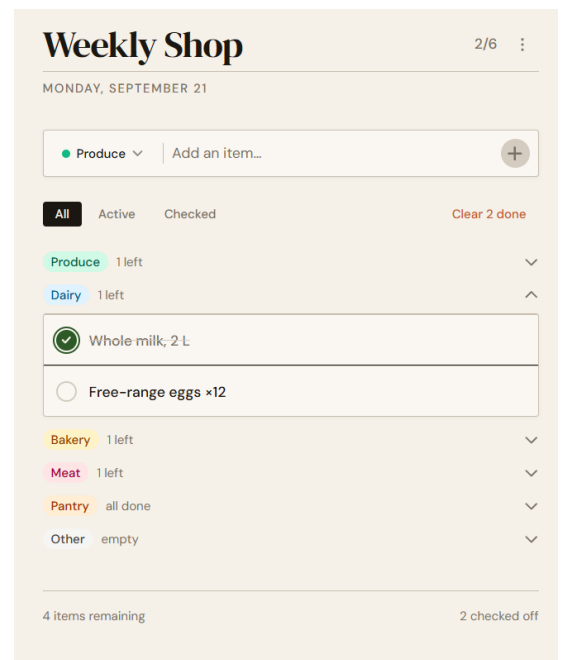
Darstellung

Wir haben in figma.com Designs mit Prompts erstellen lassen und ein design ausgewählt, das unseres Erachtens von unterschiedlich Technikaffinen Zielgruppen verwendet werden kann (**Seek universal usability**). wir haben uns für ein Helles design mit einem warmen Farbhintergrund und pastellfarbene Markierungen entschieden, da dies Beruhigend wirkt. Das Einkaufen ist stressig genug.

Danach haben wir weitere Bedingungen für die UX hinzugefügt. Z.B.: dass nur eine Gruppe gleichzeitig geöffnet sein soll, welche sich beim Öffnen einer neuen Gruppe die vorherige Gruppe schliesst. Dies dient der Übersichtlichkeit und verhindert unnötig langes scrollen. So soll ein bequemer, nach Abteilungen geordnetem Ablauf, beim Einkaufen gewährleistet werden.

Beim Abhaken eines Items wird ein grünes Häkchen gesetzt (**Offer informative feedback**) und der Status gespeichert. Was eine Filterung ermöglicht. So können z.B. nur offene Items angezeigt werden während dem Einkaufen. Oder zum Überprüfen am Ende des Einkaufs, nur die abgehakten Items. Oder aber auch alle. (**Keep users in control**)

Wir haben auf Produktbilder verzichtet und stattdessen die Gruppenfarbe wählbar gemacht. So ist zum Beispiel eine Gemüseabteilung frischgrün, eine Fleischwarenabteilung fleischrosa, eine Backwarenabteilung goldbraunReduce (**Reduce short-term memory**). Bilder nehmen auf einem Mobiltelefon zu viel platz ein, oder sie sind in einer Liste schlecht-erkennbar-klein. Deshalb wurde das Layout für den Gebrauch mit Mobiltelefon designt. Niemand nimmt das Notebook mit zum Einkaufen,



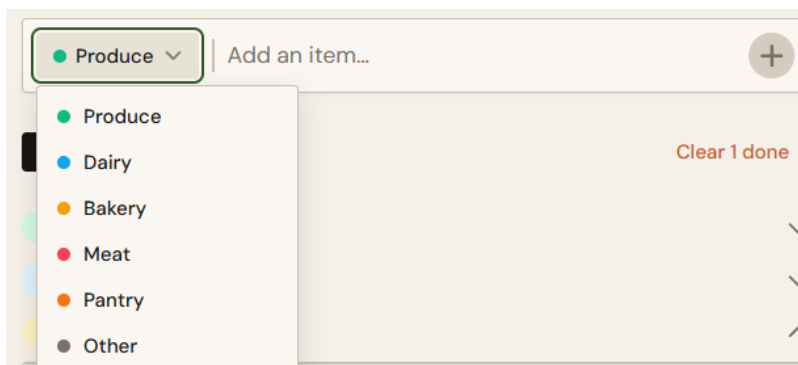
um eine Einkaufsliste abzuheben. Die 3 Buttons (Checkbox, Bearbeiten und Löschen) sind etwas klein geraten. Auch sollte das Klicken auf die item Fläche ein Abheben des Items triggern. Eine Überlegung für dessen Umsetzung wir aus Zeitgründen verzichten.

Des Weiteren können Preise pro Einheit oder Mass, und Mengen eingegeben und berechnet werden (Mehr dazu weiter unten). Dies fördert Kontrolle durch Übersicht über Ausgaben und der Nutzer erkennt Preisänderungen, wenn er die gespeicherten preise mit der Quittung vergleicht. Für Familien, die etwas mehr auf das Geld achten müssen, ist dieser kleine Mehraufwand nützlich.

Ui Elemente

Hinzufügen eines Items

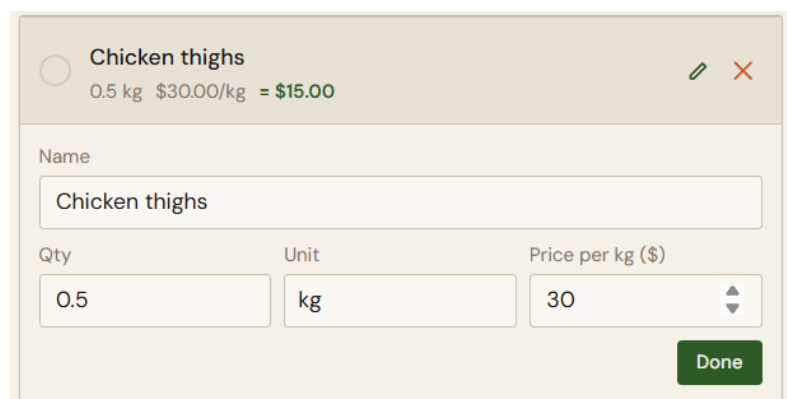
Ein Input Feld ermöglicht das Benennen des Items, und mittels Dropdownliste das Zuweisen einer Gruppe. Hinzugefügt wird das item der liste mit dem Drücken des (+) Buttons auf der rechten Seite. (**Design dialogs to yield closure**)



Preis Aufstellung

Um ein item zu bearbeiten und Preise per Einheit und Menge aufzustellen, muss der User auf das Bleistift Icon drücken (**Strive for consistency**). Hier kann der User den Namen und die Parameter für die Preisaufschlüsselung bearbeiten.

Die Input Felder Qty und Price können per Eingabe oder per Knopfdruck den wert verändern. Unit funktioniert mit einem festgelegten Dropdown und beinhaltet, Pack, Gramm, kg, Liter oder ml (**Prevent errors**).

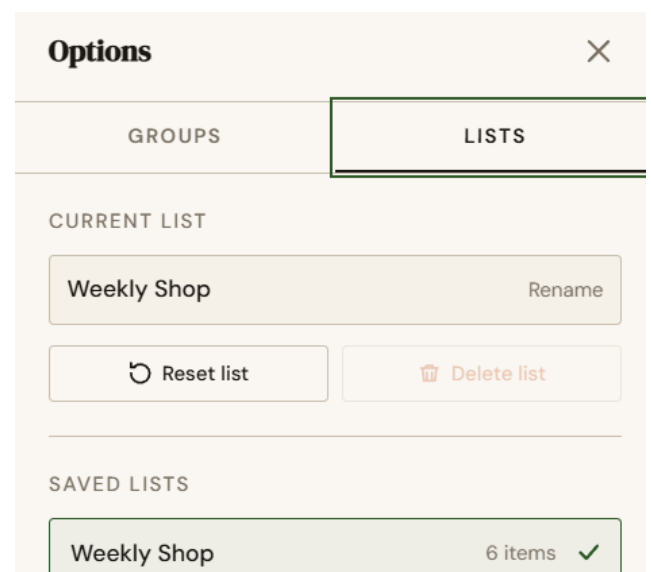
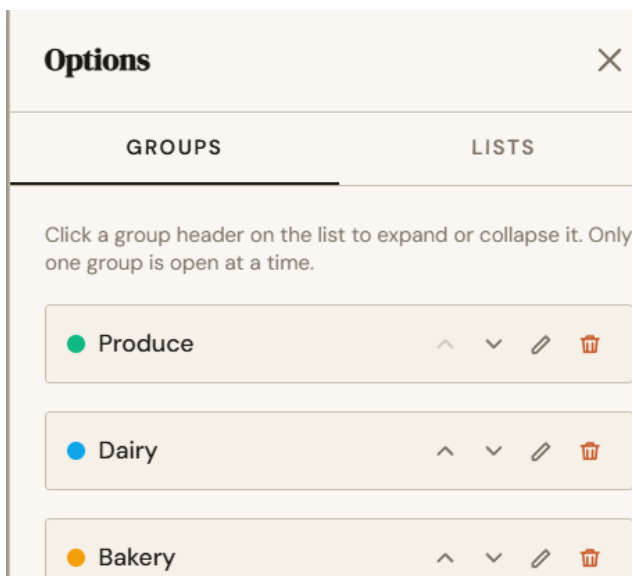


Gruppen anordnen und bearbeiten

Auf dem Options Button oben rechts,  erkennt man anhand des dreipunkte Symbol dass dies ein Optionen Menü ist (**Strive for consistency**). Im ersten Menu Groups können Gruppierungen hinzugefügt, angeordnet, bearbeitet oder gelöscht werden.

Im rechten Menu Lists können listen hinzugefügt oder gelöscht werden. Zusätzlich können listen zurückgesetzt werden, damit nicht alle Items einer Liste einzeln wieder auf offen gesetzt werden müssen. Dies soll es vereinfachen wiederkehrende listen zu verwenden, wie wöchentliche Einkaufslisten und dergleichen. Dasselbe soll geschehen, wenn eine Liste erneut geladen wird. (**Keep users in control**)

Ein Feature, das wir hinzufügen würden, wäre Items automatisch nach einer gewissen Zeit wieder aktiv zu setzen. Beispielsweise Reinigungsmittel werden nur alle 2 Monate gebraucht, Milch und Fleisch Produkte einmal wöchentlich und Brot täglich. So würde sich eine dynamisch erstellende Einkaufsliste von selbst erstellen. Items die nicht abgehakt wurden, wären farblich markiert anhand vom Dringlichkeitsstatus in Bezug auf die Zeit des Zyklus. So hätte Brot nach 3 Tagen einen hohen Dringlichkeitsstatus gegenüber einem Reinigungsmittel nach 3 tagen. Des Weiteren könnten nicht abgehackte posten auf eine andere liste exportiert werden. Z.B. eine vergessen/ nicht verfügbar liste. (**Reduce short-term memory**)



Statusleiste

Unten in der App erhält der User die Übersicht. Wie viele Produkte noch abzuhaken sind und wie viele schon abgehakt sind. Dazu eine Preis Auflistung von abgehakten und nicht abgehakten Items (**Reduce short-term memory**).

4 items remaining
Est. total (2 priced)

2 checked off
\$12.00 / \$27.00

Weitere Überlegungen

- Da einkaufen bei vielen eine Familiensache ist, könnte die App in einer Cloud geführt werden. Alle Familienmitglieder die ein Handy haben und des Einkaufens befähigt sind könnten listen bearbeiten, bzw. Items hinzufügen oder abhaken. Um Fehler zu verhindern können gespeicherte listen zwar gelöscht, aber aus dem Backup wieder hervorgeholt werden (**Permit easy reversal of actions**). Dies würde es auch ermöglichen listen bequem an einem PC zu bearbeiten, um sie dann per Handy abzuhaken.
- «*bruchsch no öppis?*» Einkaufslisten, Gruppen oder markierte Items könnten exportiert oder geteilt werden. Um mit jemandem über whatsapp oder als SMS, zu teilen.
- Detailhandelsunternehmen, Apotheken usw. könnten ihren Produkt Katalog aktuell halten und so den Usern preis Aufschlüsselung in Echtzeit ermöglichen. Mittels Handykamera könnten User den Barcode oft gekaufter Produkten der Liste oder dem Produkt Katalog hinzufügen.
- Ein Dark Modus
- Ein Finanzen-Aufschlüsselungsmodus. Mit einer KI die Zugriff auf den einkaufsverlauf, den Katalog und somit die historischen preise hat, könnte sich der User Einsparmöglichkeiten geben lassen. Auch könnte er auf Aktionen und Detailhändler mit niedrigeren preisen hingewiesen werden.
- Nach Rezept Einkaufsliste erstellen: Rezept copy-pasten, Personen Anzahl angeben und Einkaufsliste erstellen lassen.

Code - Gruppen auf- und zuklappen

Mit dieser Funktion kann ich eine Gruppe in der Einkaufsliste öffnen und wieder schliessen. Es ist immer höchstens eine Gruppe geöffnet.

Die Funktion

Zeile 159 von src/app.jsx:

```
const toggleGroup = (groupId) => {  
    setExpandedGroupId((prev) => (prev === groupId ? null :  
    groupId));  
};
```

groupId ist die ID der angeklickten Gruppe. prev enthält die ID der Gruppe, die bisher geöffnet war.

Mit `prev === groupId` prüfe ich, ob die angeklickte Gruppe bereits offen ist. Wenn ja, wird durch `? null` gespeichert und die Gruppe geschlossen. Wenn nicht, wird die ID der angeklickten Gruppe gespeichert und diese geöffnet.

Das Fragezeichen und der Doppelpunkt sind eine kurze Schreibweise für eine Wenn-sonst-Abfrage.

Mit onClick Aufrufen

Zeile 350 von src/app.jsx:

Hier wird die Funktion beim Klick auf den Gruppennamen aufgerufen:

```
<button onClick={() => toggleGroup(group.id)}>  
    {group.name}  
</button>
```

Der Button zeigt den Namen der Gruppe an. Beim Klick wird `toggleGroup` mit der ID dieser Gruppe ausgeführt. Durch `() =>` wird die Funktion erst beim Klicken aufgerufen.

Rendering

Zeile 345 von src/app.jsx

```
const isExpanded = expandedGroupId === group.id;
```

Wenn die gespeicherte ID zur Gruppe passt, ist `isExpanded = true`.

Ab Zeile 365 wird damit entschieden, ob der Inhalt angezeigt wird. Hier ist der Inhalt gekürzt:

```
{isExpanded && (  
    <div>  
        {/* Hier steht der Inhalt der Gruppe. */}  
    </div>  
)}
```

Das && bedeutet hier, dass der Inhalt nur angezeigt wird, wenn isExpanded wahr ist.

Ablauf

Der User klickt auf einen Gruppennamen. Dadurch wird toggleGroup aufgerufen und der State mit setExpandedGroupId geändert. Die Ansicht wird neu gerendert: Die geöffnete Gruppe zeigt ihren Inhalt an. Wenn der User nochmals auf dieselbe Gruppe klickt, wird sie geschlossen.

LocalStorage

In unserem Projekt haben wir keinen eigenen useLocalStorage-Hook, sondern die zwei Funktionen loadState() und saveState() in storage.js. Wenn der App gestartet wird, wird loadState() die gespeicherte Einkaufsliste aus dem localStorage geladen. Dabei wird der Schlüssel „groceries-app-v1“ verwendet.

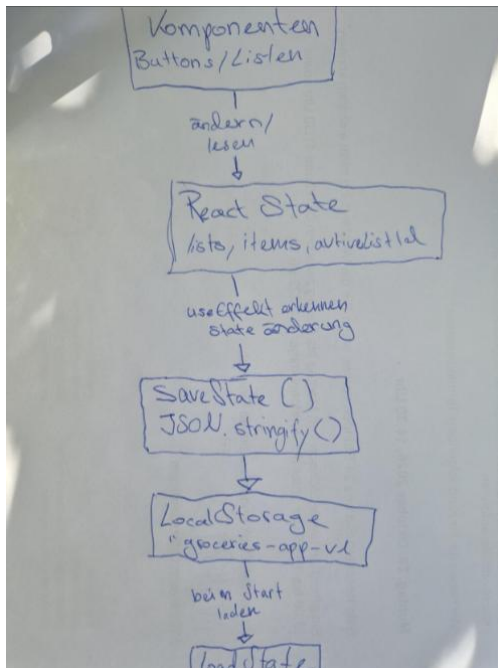
Beim Speichern wird saveState(lists, activeId) aufgerufen. Als Parameter werden die aktuellen Einkaufslisten und die ID der aktuell ausgewählten Liste übergeben. Diese werden mit JSON.stringify() in einen String umgewandelt und anschliessend mit localStorage.setItem() im Browser gespeichert.

Wenn der Benutzer auf „Produkt hinzufügen“ klickt, wird ein neues Produkt Objekt erstellt und in den aktuellen React-State der Einkaufsliste eingefügt. Wir haben einen useEffect, der auf Änderungen von lists und activeListId reagiert, indem es saveState aufruft.

Der Datenfluss ist: „Produkt hinzufügen“ → React-State wird geändert → useEffect erkennt die Änderung → saveState() → JSON.stringify() → localStorage

Um statt localStorage sessionStorage zu verwenden, müsste man in Funktionen localStorage durch sessionStorage ersetzen z.B. bei saveState() würde man localStorage.setItem durch sessionStorage.setItem ersetzen.

Technik



Was passiert, wenn zwei Produkte exakt gleich heissen?

Zwei Produkte dürfen gleich heissen, da der Produkt ID und nicht der Produktname, verwendet wird, um ein Produkt zu identifizieren.

UX/UI Testing:

Die meisten Punkte der Checkliste sind erfüllt. Unser Tester (Kieras Bruder) konnte ohne Schwierigkeiten herausfinden, wie man ein Produkt einfügt, abcheckt, bearbeitet und löscht, und schien die Status „active“ und „checked“ schnell verstanden zu haben. Die App funktioniert auf verschiedenen Bildschirmen und bei unterschiedlichen Tab-Grössen, einschliesslich Handy. Die Daten werden im lokalen Speicher abgelegt und bleiben somit auch nach einem Neustart des Browsers erhalten.

Unser Tester hatte Schwierigkeiten zu verstehen, wie man die Kategorien eines Produkts festlegt, und ging davon aus, dass der Artikel automatisch anhand seiner Eingaben sortiert würde. Als er erkannte, dass dies nicht der Fall war, versuchte er, die Kategorie über die Bearbeitungsfunktion zu ändern, was jedoch nicht funktioniert, da man dort nur den Namen, die Menge und den Preis des Produkts festlegen, aber nicht die Kategorie ändern kann.

Hätten wir mehr Zeit gehabt, hätten wir die Möglichkeit hinzugefügt, die Kategorie eines Artikels über die Bearbeitungsfunktion zu ändern, und versucht, deutlicher zu machen, wo der Nutzer die Kategorie auswählen soll. Wir könnten auch eine Option „Unsortiert“ für Nutzer hinzuzufügen, die sich nicht mit den Kategorien beschäftigen möchten, anstatt Produkte standardmässig der Kategorie „produce“ zuzuordnen. Hätten wir noch

mehr Zeit, wäre es eine interessante Herausforderung, eine Funktion hinzuzufügen, die das Produkt automatisch anhand des eingegebenen Namens kategorisiert.

Prompts

Wir haben in Figma mit folgenden Prompts eine .make Datei erstellt die uns von der Funktionalität und dem Layout zufriedengestellt hat.

Prompt 1: groceries list app

Prompt 2 : i want an options button:

grouping activated / deactivated. active group is shown the list, deactivated group is hidden only the group title is viewable. on activating a group it displays all items and hides the items of the active group before, deactivating it.

edit grouping. create / delete groups. order groups. color groups.

new list then save list, delete list, reset list (uncheck all items)

prompt 3:

add a edit putton. price, how many and unit(unit (like if you have apples 5x units), pack, kg, grams, liter, ml)

prompt 4 :

for pricing i need to calculate price per unit. qty would multiply the price

Wir haben danach die .make Datei heruntergeladen und einem Agenten gegeben zum ausführen. Leider ist der originale Prompt nicht mehr auffindbar in IntelliJ, aber er lautete in etwa wie folgt: «*Erstelle anhand dieser .make Datei eine React-App mit Vite*»